

策略交易管理系统文档

概述

系统使用 `strategy_registry` + `strategy_legs` 作为当前唯一策略配置存储，旧表 `polyMarket_Monitoring` 已降级为历史存档与兼容读取兜底，不再作为正常策略配置读写入口。

核心模块：

- `strategy_registry` + `strategy_legs`：策略注册与腿配置
- 完整 REST API（CRUD + 状态切换）
- 前端毛玻璃弹窗创建策略，策略监控首页与工作台使用统一三态 `select` 切换 `Stop` / `Virtual` / `Real`
- `StrategyCode`/ 目录存放策略代码文件
- `VirtualRunner` 已接入 `app.py` 启动链路，负责按轮询间隔运行 `state=Virtual` 的策略
- 虚拟盘五张表：`strategy_virtual_account` / `strategy_virtual_positions` / `strategy_virtual_orders` / `strategy_virtual_events` / `strategy_virtual_ticks`
- 审计表：`strategy_run_ticks` / `strategy_run_events` / `strategy_action_events`，用于记录沙盒输出、动作和错误

数据库设计

数据库路径：`Data/PolyMarketMonitoring.db`

strategy_registry（策略注册表）

列名	类型	约束	说明
strategy_id	INTEGER	PK AUTOINCREMENT	主键，同时作为 <code>row_id</code> 供旧代码引用
strategy_uid	TEXT	NOT NULL UNIQUE	UUID，外部引用标识
strategy_name	TEXT	NOT NULL	策略名称（人可读，不应填文件名）
strategy_code	TEXT	NOT NULL DEFAULT ""	关联 <code>StrategyCode</code> 目录下的文件名（不含 <code>.py</code> ）

列名	类型	约束	说明
state	TEXT	CHECK (Stop/Virtual/Real)	运行状态，替代旧 IsVirtual 字段
initial_capital	REAL	DEFAULT 0	保留字段
strategy_bankroll	REAL	DEFAULT 0	策略占用资金
profit_roll_ratio	REAL	DEFAULT 0	保留字段
realized_profit	REAL	DEFAULT 0	保留字段
input_json	TEXT	DEFAULT '{}'	策略输入参数 JSON， 展开后对应旧 Inputs1 ~ Inputs13
created_at_utc	TEXT	NOT NULL	创建时间 ISO
updated_at_utc	TEXT	NOT NULL	更新时间 ISO

当前源码的 strategy_registry 未包含 ever_real 字段；Stop / Virtual / Real 的切换限制由前端确认与后端枚举校验共同完成。

strategy_legs（策略腿表）

列名	类型	约束	说明
leg_id	INTEGER	PK AUTOINCREMENT	主键
strategy_id	INTEGER	FK → strategy_registry ON DELETE CASCADE	所属策略
leg_index	INTEGER	DEFAULT 0, UNIQUE(strategy_id, leg_index)	腿序号
condition_id	TEXT	DEFAULT ''	Polymarket condition ID
yes_token	TEXT		Yes token 地址
no_token	TEXT		No token 地址
budget_cap	REAL	DEFAULT 0	预算上限

列名	类型	约束	说明
params_json	TEXT	DEFAULT '{}'	腿级参数 JSON
yes_qty / no_qty	REAL	DEFAULT 0	当前缓存持仓数量
yes_avg_cost / no_avg_cost	REAL		当前缓存均价
yes_current_price / no_current_price	REAL		最近价格快照
unrealized_pnl	REAL	DEFAULT 0	未实现盈亏缓存
position_source	TEXT	DEFAULT ''	持仓来源标记
position_updated_at	TEXT		持仓更新时间
created_at_utc	TEXT	NOT NULL	创建时间
updated_at_utc	TEXT	NOT NULL	更新时间

Leg 身份与删除语义

- leg_id 是 SQLite 内部自增主键。删除 leg 后出现跳号是正常现象，不代表数据损坏；业务逻辑不要长期依赖旧 leg_id。
- strategy_id 是 leg 归属策略的外键。删除 strategy_registry 中的策略时，strategy_legs 会通过 ON DELETE CASCADE 同步删除。
- 当前虚拟盘持仓、订单、事件和 ticks 使用 strategy_id + leg_index 关联 leg，而不是使用 leg_id。
- PUT /api/registry/strategies/<id>/legs 会整体替换 legs。后端会先比较旧 legs 与新 legs 的身份字段：leg_index、condition_id、yes_token、no_token。
- 如果身份字段发生变化，系统会同步清理该策略的虚拟盘状态表，避免旧市场的 (strategy_id, leg_index) 持仓被误套到新市场上。
- 如果只修改 budget_cap 或 params_json，不会清理虚拟盘状态。

已移除字段

- direction：曾用于表达 Long / Short / Observe，但当前架构没有接入 PnL、预算分配或虚拟盘执行，因此已从 strategy_legs 删除。
- weight：曾作为多腿权重预留，但当前架构没有接入预算分配、PnL 加权或下单，因此已从 strategy_legs 删除。

索引

- `idx_strategy_registry_state` — `strategy_registry(state)`
- `idx_strategy_legs_strategy_id` — `strategy_legs(strategy_id)`
- `idx_strategy_legs_condition_id` — `strategy_legs(condition_id)`
- `idx_strategy_legs_yes_token` — `strategy_legs(yes_token)`
- `idx_strategy_legs_no_token` — `strategy_legs(no_token)`

旧表说明

`polyMarket_Monitoring` 和 `polyMarket_Monitor` 保留为历史存档与兼容回退来源。正常策略配置读写以 `strategy_registry` + `strategy_legs` 为准；仅在新表无数据或目标策略不存在时，部分读取路径仍会 fallback 到旧监控表。

数据流

读取路径

`_load_strategy_monitoring_rows` / `fetch_strategy_detail` 优先走新表：

```
strategy_data_source.list_strategies() / get_strategy()  
  → strategy_registry + strategy_legs  
  → strategy_to_flat_dict() (展开 input_json 为 Inputs1~13)  
  → _build_strategy_item()
```

旧表回退路径仍保留为兼容兜底：新表有数据时不会走旧表；新表为空或指定策略在新表不存在时，旧监控表仍可被读取。

写入路径

`update_strategy_detail` 优先写新表；仅当目标策略不在 `strategy_registry` 中时，才回退更新旧监控表字段：

- `initial_capital` / `profit_roll_ratio` / `realized_profit` / `strategy_bankroll` → 直接 UPDATE `strategy_registry` 对应列
- `Inputs*` 等参数 → 合并进 `strategy_registry.input_json`

API 接口

基础路径： /api

策略代码列表

GET /api/strategy-codes

返回 StrategyCode/ 目录下的 .py 文件名列表（去掉扩展名）。

GET /api/strategy-codes/<code_name>/inputs

动态加载指定策略代码，读取其 Inputs 声明，过滤 UseData 后返回可编辑参数列表，供创建/编辑弹窗自动渲染字段。

策略 CRUD

方法	路径	说明
GET	/api/registry/strategies	列出所有策略（含 legs）
POST	/api/registry/strategies	创建策略
GET	/api/registry/strategies/<id>	获取单个策略详情
PUT	/api/registry/strategies/<id>	更新策略
PATCH	/api/registry/strategies/<id>/state	切换状态（Stop/Virtual/Real）
PUT	/api/registry/strategies/<id>/legs	替换策略腿；若 leg 身份字段变化，会清理该策略旧虚拟盘状态
DELETE	/api/registry/strategies/<id>	删除策略（CASCADE 删除腿）

虚拟盘 API

方法	路径	说明
GET	/api/virtual/strategies/<id>/account	虚拟账户状态（cash/equity/pnl/fees）
GET	/api/virtual/strategies/<id>/positions	虚拟持仓列表（按 leg_index + side）
GET	/api/virtual/strategies/<id>/orders	虚拟订单流水（支持分页）

方法	路径	说明
GET	/api/virtual/strategies/<id>/events	事件流（actions/print，支持 event_type 过滤）
GET	/api/virtual/strategies/<id>/ticks	Tick 运行日志列表
POST	/api/virtual/strategies/<id>/reset	重置虚拟账户（清空持仓/订单/账户，保留策略配置）

创建策略 POST 请求体示例

```
{
  "strategy_name": "NVDA 市值策略",
  "strategy_code": "Stragy_Fllow_Stock_Value",
  "state": "Virtual",
  "strategy_bankroll": 20,
  "condition_id": "0x...",
  "yes_token": "0x...",
  "no_token": "0x...",
  "budget_cap": 20,
  "input_json": { "AnchorCompany": "NVDA", "RankPosition": "1" }
}
```

状态切换 PATCH 请求体

```
{ "state": "Real" }
```

状态机设计

state	含义	颜色
Stop	不运行	灰色 / slate
Virtual	虚拟盘运行	浅蓝色
Real	实盘运行	浅绿色

切换入口：

- 策略监控首页 Mode 列。
- 单策略工作台 header 状态 select。

两个入口必须保持同一套状态、配色、确认文案和保存逻辑。保存接口统一为：

```
PATCH /api/registry/strategies/<id>/state
```

请求体：

```
{ "state": "Real" }
```

切换路径：当前源码允许三态两两切换； `strategy_registry_service.update_strategy_state()` 只校验目标值必须属于 `Stop / Virtual / Real`。

迁移与副作用规则：

- `Virtual -> Real`：弹窗提醒；虚拟仓位、虚拟订单、虚拟 PnL 不迁移到真实账户。
- `Real -> Virtual`：弹窗提醒；真实仓位和真实挂单不自动迁移、不自动撤单、不自动平仓。
- `Stop -> Virtual`：进入虚拟盘调度范围，后续由 `VirtualRunner` 执行。
- `Virtual -> Stop`：退出虚拟盘调度范围，不清空历史 Print / Action。
- `Stop -> Real`：进入实盘模式入口，使用真实账户状态。
- `Real -> Stop`：停止策略模式，不自动处理真实挂单或仓位。

调度规则：`VirtualRunner` 只读取 `state=Virtual` 的策略。`Stop` 与 `Real` 不进入虚拟盘调度循环。

文件结构

```
polymarket_datatube/
├─ app.py                                # Flask 路由
├─ migrate_to_strategy_tables.py         # 一次性迁移脚本（已执行，保留备查）
├─ StrategyCode/                         # 策略代码文件目录（.py）
├─ Data/
│   └─ PolyMarketMonitoring.db          # SQLite 数据库
├─ services/
│   ├── strategy_registry_service.py     # 策略 CRUD 服务层
│   ├── strategy_data_source.py          # 统一读写层（含虚拟盘 DDL）
│   ├── polymarket_service.py            # 监控服务（读写均走新表）
│   ├── strategy_settings_service.py     # 策略设置服务（state 替代 IsVirtual）
│   ├── strategy_audit_store.py          # 策略运行审计表
│   ├── order_store.py                  # 实盘订单状态机与订单事件
│   ├── market_deltas_cleanup.py         # market_deltas 保留期清理工具
│   ├── virtual_runner.py                # 虚拟盘调度循环
│   ├── virtual_execution.py             # 虚拟成交引擎
│   └─ virtual_context_builder.py        # UseData 上下文注入层
├─ static/
│   ├── app.js
│   ├── settings.js
│   ├── strategy_workspace_v2.js
│   ├── workspace_v3_patch.js
│   ├── workspace_v3.css
│   └─ styles.css
└─ templates/
    ├── index.html
    └─ settings.html
```

虚拟盘数据表

五张表在 strategy_data_source._DDL_VIRTUAL 中定义，connect() 时幂等建表。

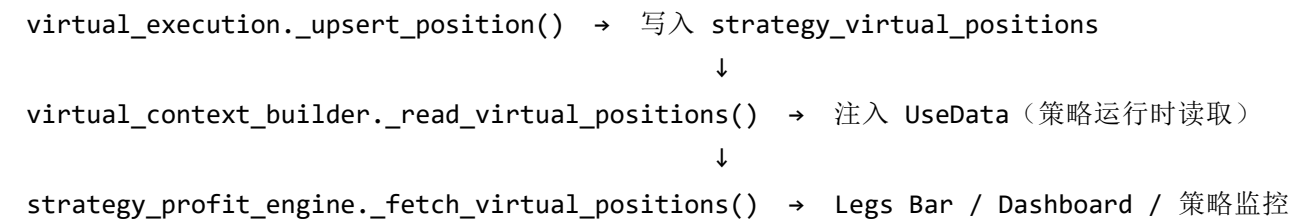
表名	说明
strategy_virtual_account	每策略一行，记录虚拟现金/权益/PnL/累计手续费

表名	说明
strategy_virtual_positions	按 (strategy_id, leg_index, side) 唯一，平均成本法； 当策略腿身份变化时会被同步清理
strategy_virtual_orders	虚拟订单流水，含手续费字段，status = filled / blocked / failed
strategy_virtual_events	策略事件流（print / error / settle 以及历史 action 展示事件）， 支持相邻去重
strategy_virtual_ticks	每次调度运行日志，记录 FunctionJson 原始输出与执行结果

Virtual 模式持仓数据源

Virtual 模式下，持仓数据的唯一真实来源是 strategy_virtual_positions 表。

数据流：



strategy_profit_engine.compute_and_persist_strategy_profit() 在遍历策略时检查
state == 'Virtual'：

- Virtual 策略：从 strategy_virtual_positions 读 qty / avg_price，position_source = "virtual"
- Real 策略：从钱包 API + 本地订单库读持仓（原有逻辑不变）

这确保工作台 Legs Bar（Yes Qty / No Qty）、Dashboard（EXPOSURE / PNL）和图表
（yes_position）三处数据源一致。

手续费公式

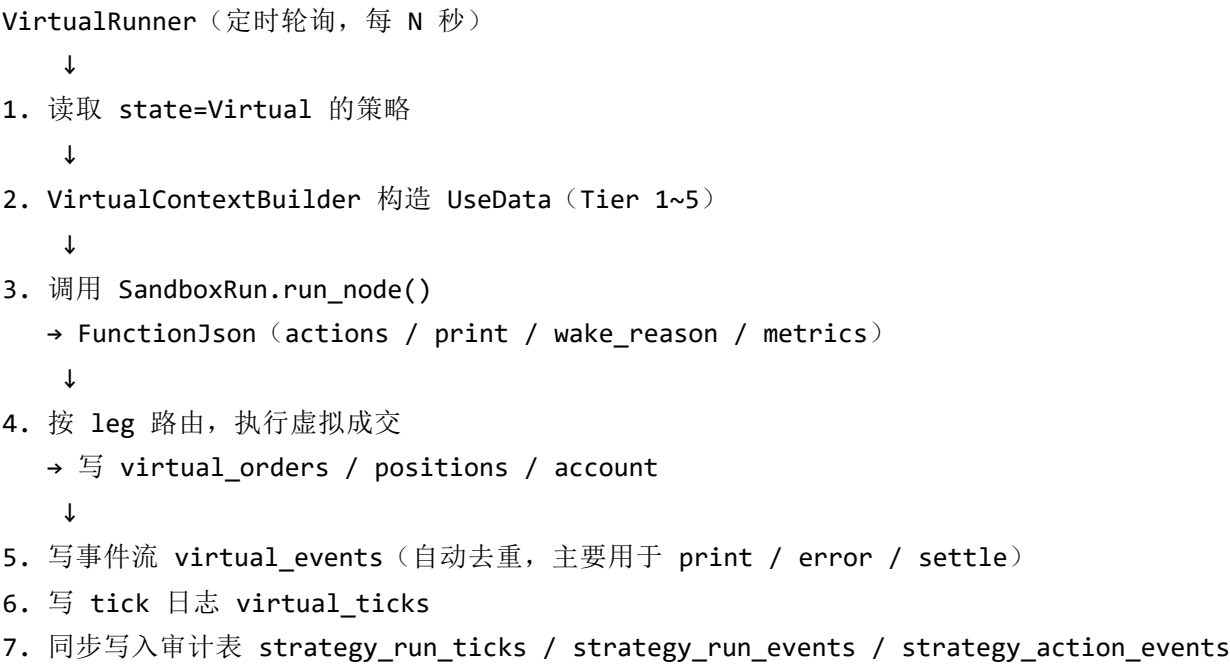
$$\text{fee} = \text{qty} \times \text{fee_rate} \times \text{price} \times (1 - \text{price})$$

费率：Crypto=0.072，Politics/Finance/Tech=0.04，Sports=0.03，其余=0.05

注意：执行层已保留按 market_category 映射费率的能力；当前 virtual_runner.py 尚未向执行层注入
市场类别，因此未传类别时实际使用默认费率 0.05。

虚拟盘运行系统

整体调度流程



UseData 注入优先级

- Tier 1 盘口（_L{n} 后缀）
- Tier 2 预算派生（Yes/No_Max/Min_BudgetCap）
- Tier 3 时间（NowTime / Enddate / day_to_end / hour_to_end）
- Tier 4 外部行情（Price_{SYMBOL} / McapUsd_{SYMBOL} 等）
- Tier 5 input_json 用户自定义参数

FunctionJson 动作

策略代码的完整输入、UseData 命名、FunctionJson 多动作协议和第一阶段落地范围见 [strategy-code-spec.md](#)。本节仅保留运行系统层面的摘要说明。

函数	说明
Buy(side, leg, qty, price=None)	买入，price=None 时用盘口 ask
Sell(side, leg, qty, price=None)	卖出，price=None 时用盘口 bid

函数	说明
SetPosition(side, leg, pct)	调仓至目标比例（相对 BudgetCap）

序列化格式：

```
{
  "actions": [
    {"type": "BUY",    "side": "Yes", "leg": 0, "qty": 10},
    {"type": "SELL",   "side": "No",  "leg": 0, "qty": 5, "price": 0.35},
    {"type": "SETPOS", "side": "Yes", "leg": 1, "pct": 0.3}
  ],
  "print": ["..."],
  "wake_reason": null
}
```

资金不足时写 status='blocked' ， reason='insufficient_cash' ，不中断其他策略。

事件流

事件来源

工作台事件流（ /api/polymarket/strategies/<row_id>/events ）聚合两类来源：

1. **虚拟盘事件**：来自 strategy_virtual_events 表，包含 print （策略输出）、error （执行错误）、settle （结算）以及历史 action 展示事件。
2. **Actions 事件**：strategy_event_service.py 从 strategy_action_events 审计表读取策略动作，转换为 event_type=action 事件返回。它展示策略原始动作（如 SETPOS ）及执行结果（如 -> BUY 、 blocked 、 already_at_target ）。
3. **Trades 事件**：strategy_event_service.py 从 strategy_virtual_orders 读取 status='filled' 的实际成交订单，转换为 event_type=trade 事件返回。blocked / failed 属于动作执行结果，展示在 Actions 中，不混入 Trades。

前端渲染稳定性

事件流前端维护一个 _fullEventsList 全局缓存：

- loadEvents() （轮询拉取）写入完整列表。

- `appendWorkspaceEvent()`（SSE 推送）将新事件合并进完整列表，而不是替换。
- 工作台启动时，`loadEvents()` 必须在 `/workspace` 基础信息加载后立即独立触发，不能等待 `/chart` 完成；否则图表慢或失败时，事件流标签会停留在空 DOM。
- 事件流默认接口返回“最近 `limit` 条 + 每个关键类型最多 20 条保底事件”，避免高频 `print` 挤掉 `trade / action / error / settings`。
- 前端 `_fullEventsList` 也必须按同样规则裁剪；SSE 每 3 秒只推最新单条事件，不能对合并后的列表做简单 `.slice(0, 120)`，否则保底的旧 `trade` 会被最新 `print` 挤掉，表现为 `Trades` 标签周期性消失又回来。

这消除了 `Trades` 标签在 SSE 推送时数据闪现消失的问题。

签名去重按当前过滤器计算，避免 `print` 事件 `ts` 变化触发无效重渲染。

事件类型过滤

标签	过滤条件
All	全部
Print	<code>event_type=print</code>
Actions	<code>event_type=action</code>
Trades	<code>event_type=trade</code>
Errors	<code>event_type=error</code>

图表 Events 面板与 print 事件

工作台图表中的事件分为两种展示语义：

- **主图事件竖线**：只用于 `action / trade / error / settings` 等需要定位到价格图上的关键事件。
- **Events 面板事件点**：用于完整展示事件流，包括高频 `print`。

`print` 是策略解释、调试和审计输出，频率通常远高于 `action / trade`。它必须继续由 `/api/polymarket/strategies/<row_id>/chart?include_events=1` 返回，并继续显示在 Events 面板里；但前端不应把 `print` 画成主图 `markLine` 竖线，否则会在价格主图右侧形成密集竖线墙，干扰盘口与 PNL 判断。

前端实现约定：

- `buildEventMarkLines()` 必须过滤 `print`，只生成主图竖线。
- `buildEventTimelineSeries()` 必须使用原始 `payload.events`，不能复用主图竖线过滤结果。

- 图表结构签名必须把 event_timeline 的出现/消失纳入结构判断；否则当某个时间窗只有 print 事件时，ECharts 可能沿用旧结构，导致 Events 面板不创建。
- 模板中 strategy_workspace_v2.js 的 query version 需要随此类前端逻辑变更更新，避免浏览器缓存继续运行旧图表逻辑。

排查口径：如果主图没有 print 竖线，这是预期；如果 Events 面板也没有 print，先查 chart 接口 events 是否返回 type=print ，再查前端是否请求了 include_events=1 、静态 JS 版本号是否已刷新。

PNL 计算

价格来源优先级

工作台 PNL 与 Dashboard PNL 使用同一套价格来源逻辑：

1. CLOB orderbook REST （ services/clob_orderbook_service.py 调 /book ）是策略监控、策略工作台、Virtual UseData 的优先盘口来源。
2. 本地实时库 （ markets_state / market_deltas ）是可观测的实时层；WebSocket 正常时由 WS 写入，WS 不可用时由 REST fallback 写入 book_rest_fallback 事件保鲜。
3. Gamma / 字典市场快照只作为市场身份、标题、slug、到期时间等元数据来源；只有 CLOB 与本地实时库都不可用时，才允许作为最后价格 fallback。

本地缓存文件 polymarket_active_markets_cache.json 有新鲜度上限，旧文件不能因为被读取就刷新内存时间戳。否则会出现“本地看似有价，但官网盘口已经变了”的错配。

workspace_fast_path 传入 _match_strategy_market 的结果，策略监控首页、工作台摘要、Legs Snapshot 和图表当前点都从同一套当前盘口对象取值，避免 Dashboard / Workspace 使用不同价格源。

WebSocket 保鲜与健康检查

不能只看 thread_alive=true 判断本地 WS 健康。 /api/health 的 ws_market_sync.monitor 才是排障入口：

字段	含义
token_count / shard_token_counts	当前订阅 token 数；Web 进程只订阅策略/持仓显式 token，不能混入大范围概率扫描
shard_connected	分片连接状态

字段	含义
last_subscribe_at	最近一次实际发送 subscribe 的时间
last_msg_at	最近一次收到 WS 消息或 REST fallback 刷新的时间
last_update_at	最近一次写入 markets_state / market_deltas 的时间
last_ws_error	最近 WS 错误
msg_count / update_count	收到消息和落库更新计数

如果 thread_alive=true 但 last_update_at 长时间不变，说明线程活着但本地行情已过期。分片重连后必须重新订阅当前 token 集；WS 握手失败时，REST fallback 会刷新当前分片 token，避免本地库长期停在旧盘口。

信息错配防线

- 二元市场优先分别读取 Yes token 与 No token 的 orderbook；只有缺少某一侧盘口时才用 1 - 对侧价格 推导。
- yes_mark / no_mark 与 PNL 使用同一当前 ask 口径，不能用 Gamma outcomePrices 或 strategy_legs.yes_current_price 旧缓存覆盖。
- strategy_legs.yes_current_price / no_current_price 是兼容字段，不是权威行情源。
- 图表历史可以读 market_deltas，但当前价需要叠加 fetch_strategy_detail() 的当前盘口点。

Virtual 模式 PNL 公式

Virtual 模式必须区分“持仓浮盈亏”和“账户总盈亏”。工作台和 Dashboard 顶部的 strategy_pnl 使用账户总盈亏，避免策略平仓后已实现亏损和手续费因为 qty=0 被显示成 0。

账户总盈亏：

```
equity = cash + liquidation_value - estimated_exit_fees
strategy_pnl = virtual_total_pnl = equity - initial_cash
```

其中 liquidation_value 使用当前可卖出盘口估值：

```
yes_liquidation_value = yes_qty * current_yes_bid
no_liquidation_value  = no_qty  * current_no_bid
```

如果当前 bid 缺失，允许临时回退到同侧 ask 或持仓均价，但返回字段应继续标明 pnl_source ，方便排查。

手续费沿用 Polymarket 费用公式：

$$\text{fee} = \text{qty} \times \text{fee_rate} \times \text{price} \times (1 - \text{price})$$

strategy_virtual_account.realized_pnl 只记录成交价差； total_fees_paid 单独记录已支付手续费。因此完整账户收益应理解为：

$$\text{virtual_total_pnl} = \text{realized_pnl} + \text{virtual_unrealized_pnl} - \text{total_fees_paid}$$

其中 virtual_unrealized_pnl 使用当前可卖出价并扣除预估退出手续费：

$$\text{virtual_unrealized_pnl} = \text{liquidation_value} - \text{open_cost} - \text{estimated_exit_fees}$$

API 摘要字段：

字段	含义
strategy_pnl / virtual_total_pnl	Virtual 账户总盈亏，工作台和 Dashboard 的主 PnL
virtual_realized_pnl	已平仓价差盈亏，不含手续费
virtual_unrealized_pnl	当前持仓按可卖出价估值后的浮盈亏，含预估退出手续费
virtual_fees_paid	已支付手续费
virtual_cash	虚拟账户现金
virtual_equity	虚拟账户权益
pnl_source	PnL 口径来源，例如 virtual_account_equity

图表行级浮盈亏仍可使用可见 ask 口径，用于保持历史曲线和同一行可见价格一致：

$$\begin{aligned} \text{unrealized_pnl} &= (\text{current_ask} - \text{avg_price}) * \text{yes_qty} \\ &\quad + (\text{current_ask_no} - \text{avg_price_no}) * \text{no_qty} \end{aligned}$$

current_ask 优先取 CLOB /book 的当前 ask；本地实时库作为 WS/REST fallback 后的可观测副本。

Chart / Virtual Tick 价格防污染规则

2026-05-13 起，策略运行与工作台图表对 bid/ask 的处理遵循以下防污染规则。

Virtual tick 持久化真实 UseData 盘口

services/virtual_runner.py 在每轮虚拟策略运行结束后，会把本轮策略实际收到的 UseData 盘口写入 strategy_virtual_ticks.mode_output.price_snapshot：

```
{
  "price_snapshot": {
    "yes_bid": "...",
    "yes_ask": "...",
    "no_bid": "...",
    "no_ask": "..."
  }
}
```

字段来源优先使用：

- L0_Yes_BidPrice / L0_Yes_AskPrice
- L0_No_BidPrice / L0_No_AskPrice
- 兼容旧 key：Yes_BidPrice、Yes_AskPrice、Yes_now_bid、Yes_now_ask 等。

这样 chart 可以读取结构化 ask，而不是从策略 print 文本里猜。

print 不是完整盘口

策略 print 常见格式：

```
[INPUT]
Yes_bid=0.82 No_bid=0.17
```

这只是策略解释输出，不是完整 orderbook。Chart 可以把它作为历史 bid fallback，但不得从 No_bid 推导 Yes Ask = 1 - No_bid。过去这种推导会导致真实 ask 不变时，合成 ask 插入时序，进而让 PnL 出现锯齿。

0 报价是缺价，不是市场价

工作台图表只接受二元市场的有效价格区间 $(0, 1]$ 。 `services/strategy_chart_service.py` 中的 `_safe_binary_quote()` 会把以下值视为缺失报价：

- `None`、空字符串、非数字、`NaN / Infinity`
- `0` 或负数
- 大于 `1` 的值

这个规则同时作用于：

- `strategy_virtual_ticks.mode_output.price_snapshot`
- `function_json.print` 中解析出的 `Yes_bid=...` `No_bid=...` 历史 bid fallback
- `fetch_strategy_detail()` 当前详情快照
- 本地实时库 `market_deltas / markets_state` 中的 `bid / ask / last`

背景：Virtual tick 有时会在盘口暂时不可用时输出 `Yes_bid=0.0 / No_bid=0.0`，这表示“本轮没有有效盘口”，不是官网图表上的真实价格。如果把它写进 chart rows，ECharts 会从正常价格画一根竖线到 0，形成插针；Virtual PnL 也会按 0 标记持仓，产生不真实的瞬时亏损。

正确行为是：这些 0 报价被过滤掉，由后续行级 carry-forward 展示上一条真实盘口，或在没有上一条真实盘口时留空。

PnL 与可见 ask 同步

图表中的 `strategy_pnl` 必须由同一行可见 ask 计算：

```
yes_pnl = (market_0_yes_ask - yes_avg) * yes_qty
no_pnl  = (market_0_no_ask  - no_avg)  * no_qty
```

如果某行没有对应 ask，则该行不应显示由 bid、last 或合成 ask 推出来的 PnL。

首屏与增量渲染一致

工作台前端对 `/chart` 首屏全量 rows 和 `/chart-delta` 增量 rows 使用同一行级 carry-forward 语义。首屏加载时会在 Debug 输出 `[WS] chart:normalize-rows`，用于确认 Yes Ask / No Ask 是否在进入 ECharts 前已按最新已知真实报价延续。

这解决的是渲染口径不一致，不改变底层报价来源；真实 ask 仍必须来自 `price_snapshot`、本地 `market_deltas / markets_state` 或 CLOB `/book`。

2026-05-19 多资产架构补充

策略系统新增一层多资产抽象，目标是让同一个策略可以同时绑定 Polymarket、crypto、股票与股票期权等标的。当前实现保持旧 Polymarket 策略兼容。

Params / UseData / State 边界

类型	存储或生成位置	说明
Params	strategy_registry.input_json / strategy_legs.params_json	用户配置参数，低频修改
UseData	virtual_context_builder.build_use_data() 每轮生成	本轮运行事实快照，只读
RuntimeState	strategy_state.namespace = runtime	策略内部持久状态，由 state_updates 写回
Controls / UserState	strategy_state.namespace = user	用户人工干预状态，由 Dashboard/API 写入， 默认值来自策略代码 ControlsSchema
SystemState	strategy_state.namespace = system	系统预留状态，策略只读

新增 strategy_state 表，策略通过 FunctionJson.state_updates 写回 RuntimeState ，下一轮由 UseData["RuntimeState"] 注入。 UseData["State"] 暂时保留为 RuntimeState 的兼容别名。用户可通过 Dashboard 策略表的 State 按钮编辑 Controls override；系统按策略代码 schema 默认值 + override 合并。

strategy_legs 通用标的字段

strategy_legs 新增：

```
asset_class
venue
symbol
instrument_id
instrument_json
```

Polymarket leg 默认：

```
asset_class = polymarket_binary
venue = polymarket
instrument_id = poly:condition:<condition_id>
```

Crypto / 股票 / 期权可以使用：

```
crypto:binance:BTCUSDT
equity:US:NVDA
option:US:AAPL:20260619:250:C
```

UseData v2

UseData 现在包含结构化字段：

```
Params
RuntimeState
Controls
UserState
SystemState
State
Portfolio
Instruments
```

旧扁平字段仍保留，例如 L0_Yes_AskPrice、Yes_now_ask、Price_NVDA。

v2 虚拟盘

新增：

```
strategy_virtual_positions_v2
strategy_virtual_orders_v2
strategy_cash_ledger
```

非 Polymarket 的 ORDER / SET_TARGET 会写入 v2 表。旧 Polymarket SETPOS / BUY / SELL 继续写入原有虚拟盘表，避免工作台和图表被破坏。

完整说明见 [multi-asset-strategy-architecture.md](#) 与 [strategy-state-store.md](#)。